

Insercao de registros unicos em CSV

Comparacao entre busca sequencial e busca binaria com MergeSort

Disciplina: Analise de Algoritmos

Linguagem: C (C11)

Identificador adotado: coluna Identificador

Repositorio Git publico: <https://github.com/MatheusSZR/atividade-csv-unico>

Objetivo: garantir que novas entradas sejam acrescentadas ao final do arquivo de destino sem repetir o identificador de um registro existente ou de outra entrada nova.

Fontes de dados

Portal de Dados Abertos - conjunto "Compras e contratos": dados.gov.br/dados/conjuntos-dados/compras_contratos

Consulta publica de contratos utilizada para observar a ordem dos identificadores:
compras.dados.gov.br/contratos/v1/contratos?cnpj_contratada=05513573000132

1. Introdução

O programa foi desenvolvido de duas formas para verificar se o identificador de um novo registro já existe no arquivo CSV de destino. Na primeira forma, a verificação é feita diretamente no arquivo usando busca sequencial. Na segunda, os identificadores do destino são carregados em um vetor na RAM, esse vetor é ordenado por MergeSort e a verificação passa a usar busca binária.

Para evitar duplicatas, cada entrada nova é verificada antes de ser adicionada ao arquivo. Quando uma entrada é aceita, sua chave também passa a fazer parte das chaves já conhecidas. Com isso, uma repetição dentro do próprio arquivo de novas entradas também é recusada.

Foram disponibilizados quatro modos para executar e comparar as implementações: seq-it, seq-rec, merge-it e merge-rec. Os dois primeiros usam busca sequencial, e os dois últimos usam MergeSort e busca binária.

2. Arquivo CSV escolhido

Para a atividade, foi usado como referência um conjunto de contratos governamentais disponibilizado no Portal de Dados Abertos. O portal possui recursos em CSV relacionados a contratos, e a consulta pública de Compras Governamentais apresenta a coluna "Identificador do Contrato".

Para verificar a ordem dos valores, foi usada a consulta filtrada pelo CNPJ 05.513.573/0001-32. A página pública exibe 16 contratos e, no início da listagem, aparecem os identificadores 17021450000032011, 78481050000052008, 15851550000082013, 17021750000122009 e 15306350000132008. Esses valores já mostram que a coluna não está organizada em ordem crescente nem decrescente.

No programa, a coluna Identificador é usada como a chave única. A verificação dessa chave acontece antes de cada inserção.

3. Requisitos funcionais da solução

Item	Implementação
Entrada 1	Nome do CSV contendo novas entradas
Entrada 2	Nome do CSV de destino
Chave	Coluna Identificador, localizada pelo cabeçalho
Ação	Inserir novas linhas no final do destino
Regra	Não inserir uma chave já existente ou repetida nas novas entradas
Demonstração	Quatro modos: sequencial iterativo/recursivo e MergeSort + binária iterativo/recursivo

Comando principal:

```
./csv_unico <novas.csv> <destino.csv> [modo]
```

O terceiro argumento é opcional e existe para demonstrar separadamente as quatro implementações. Quando omitido, o modo padrão é merge-it.

4. Solucao 1 - busca sequencial

Na Solucao 1, o arquivo de destino continua nao ordenado. Para cada nova entrada, a chave e procurada registro por registro. A busca comeca novamente no inicio do arquivo para incluir tambem as entradas que foram adicionadas durante a mesma execucao.

4.1 Busca sequencial iterativa

A implementacao percorre as linhas do CSV com um laco. Em cada linha valida, o campo Identificador e extraido e comparado com a chave procurada. Se houver igualdade, a linha nao e inserida; se o arquivo chegar ao fim sem encontrar a chave, a nova linha e adicionada.

Complexidade temporal por busca: melhor caso $O(1)$; caso medio $O(n)$; pior caso $O(n)$.

Complexidade espacial extra: $O(1)$, pois nao ha pilha de recursao nem vetor proporcional ao numero de registros.

4.2 Busca sequencial recursiva

A versao recursiva faz a mesma verificacao, mas uma chamada analisa uma linha e chama a proxima para continuar a busca. Ao chegar ao fim do arquivo, a funcao retorna falso. Foi usado um unico buffer estatico, evitando criar um novo buffer a cada chamada.

Complexidade temporal por busca: melhor caso $O(1)$; caso medio $O(n)$; pior caso $O(n)$.

Complexidade espacial extra: $O(n)$ no pior caso, devido ao numero de chamadas pendentes na pilha.

4.3 Comparacao entre as duas versoes

Versao	Melhor	Medio	Pior	Espaco extra
Iterativa	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Recursiva	$O(1)$	$O(n)$	$O(n)$	$O(n)$ - pilha

As duas versoes apresentam o mesmo crescimento de tempo. A diferenca principal esta no uso de memoria: a versao recursiva precisa manter as chamadas pendentes na pilha.

5. Solução 2 - MergeSort + busca binária

Na Solução 2, primeiro os identificadores do arquivo de destino são carregados na RAM. Depois, o vetor é ordenado com MergeSort. Com os valores ordenados, a verificação de cada nova chave é feita com busca binária.

5.1 MergeSort recursivo

O MergeSort recursivo divide o vetor em duas partes até chegar a blocos de um elemento e, depois, intercala as partes já ordenadas. O vetor auxiliar é criado uma vez e reutilizado nas intercalações.

Tempo: $O(n \log n)$ no melhor, médio e pior caso. Espaço auxiliar: $O(n)$ para o vetor de apoio, mais $O(\log n)$ de pilha. Como $O(n)$ domina $O(\log n)$, o espaço assintótico total continua $O(n)$.

5.2 MergeSort iterativo

A versão iterativa não usa recursão. Ela começa com blocos de tamanho 1 e dobra esse tamanho a cada rodada: 1, 2, 4, 8 e assim por diante, intercalando os blocos vizinhos em cada etapa.

Tempo: $O(n \log n)$ em todos os casos. Espaço auxiliar: $O(n)$. Espaço de pilha adicional: $O(1)$.

5.3 Busca binária iterativa

A busca binária é feita sobre um vetor ordenado. Em cada passo, a chave é comparada com o elemento central e uma das metades do intervalo é descartada. Por isso, o número de posições analisadas cresce de forma logarítmica.

Tempo: melhor caso $O(1)$; médio $O(\log n)$; pior caso $O(\log n)$. Espaço extra: $O(1)$.

5.4 Busca binária recursiva

A versão recursiva segue a mesma ideia da busca binária iterativa: compara com o elemento central e continua na metade esquerda ou direita, de acordo com o resultado da comparação.

Tempo: melhor caso $O(1)$; médio e pior caso $O(\log n)$. Espaço extra: $O(\log n)$ de pilha.

5.5 Complexidades da Solução 2

Algoritmo	Tempo	Espaco extra	Observação
MergeSort iterativo	$O(n \log n)$	$O(n)$	Sem recursão
MergeSort recursivo	$O(n \log n)$	$O(n) + O(\log n)$	Pilha recursiva
Busca binária iterativa	$O(\log n)$	$O(1)$	Vetor ordenado
Busca binária recursiva	$O(\log n)$	$O(\log n)$	Pilha recursiva

Depois de aceitar uma nova entrada, o vetor usado na Solução 2 precisa continuar ordenado. Por isso, a nova chave é colocada na posição correta e os valores seguintes são deslocados. Essa operação pode custar $O(n)$ por inserção. Assim, para k entradas novas, a análise do processamento completo pode chegar a $O(n \log n + k \log n + kn)$, embora a busca isoladamente continue sendo $O(\log n)$.

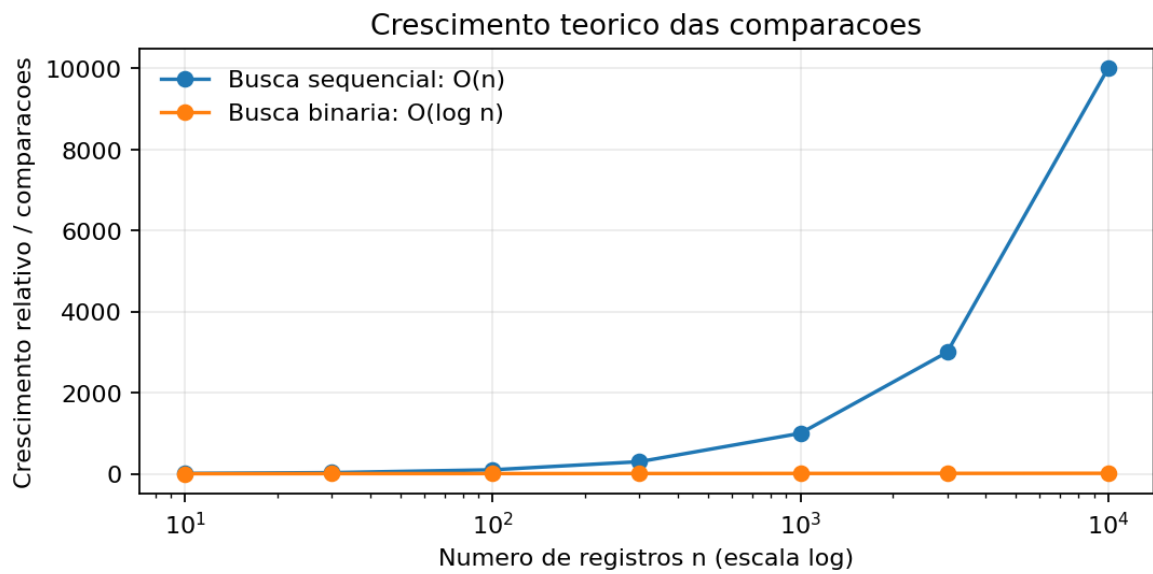


Figura 1 - Comparação do crescimento teórico das comparações entre busca sequencial e busca binária.

6. Comparação entre Solução 1 e Solução 2

A diferença mais clara entre as duas soluções está na forma de fazer a busca. Na Solução 1, o programa percorre o arquivo não ordenado e pode verificar n registros. Na Solução 2, depois da ordenação, cada comparação elimina metade do intervalo de busca.

A Solução 2 também precisa manter as chaves na RAM e executar uma ordenação inicial. Quando uma nova chave é aceita, o vetor precisa ser atualizado para continuar ordenado. Assim, além do custo da busca, a Solução 2 tem custos extras de preparação e manutenção.

6.1 Visão consolidada

Aspecto	Solução 1	Solução 2
Estrutura durante a busca	Arquivo CSV não ordenado	Vetor de chaves ordenado na RAM
Preparação	Nenhuma ordenação	MergeSort: $O(n \log n)$
Busca iterativa	Sequencial: $O(n)$ no pior caso	Binária: $O(\log n)$ no pior caso
Busca recursiva	Sequencial: $O(n)$ + pilha $O(n)$	Binária: $O(\log n)$ + pilha $O(\log n)$
Memória principal	$O(1)$ extra na versão iterativa	$O(n)$ para armazenar as chaves e auxiliar do MergeSort
Atualização do vetor	Não se aplica	Inserção ordenada pode custar $O(n)$ por nova chave

6.2 Crescimento para k entradas

Quando várias entradas são processadas, o custo aumenta conforme o tamanho atual do destino. Na Solução 1, a quantidade de comparações cresce com o número de entradas e com o tamanho do arquivo. Na Solução 2, as buscas continuam menores, mas existe o custo de manter o vetor ordenado a cada nova inserção.

Assim, a busca binária reduz o custo das consultas quando os dados já estão ordenados, mas o custo total da Solução 2 também inclui a ordenação inicial e a atualização do vetor.

7. Testes e demonstracao

Para testar as quatro implementacoes, foi usado um conjunto pequeno de dados controlados. O arquivo de destino inicial possui 5 identificadores nao ordenados. O arquivo de novas entradas possui 4 linhas: 2 chaves novas, 1 chave que ja estava no destino e 1 chave repetida de uma entrada nova anterior.

Comandos executados:

```
./csv_unico data/novas_exemplo.csv destino_seq_it.csv seq-it
./csv_unico data/novas_exemplo.csv destino_seq_rec.csv seq-rec
./csv_unico data/novas_exemplo.csv destino_merge_it.csv merge-it
./csv_unico data/novas_exemplo.csv destino_merge_rec.csv merge-rec
```

Resultado consolidado:

Modo	Lidas	Inseridas	Duplicadas rejeitadas	Comparacoes
seq-it	4	2	2	19
seq-rec	4	2	2	19
merge-it	4	2	2	9
merge-rec	4	2	2	9

Depois da execucao, o arquivo possui os 5 registros originais e as 2 novas entradas. As chaves duplicadas foram recusadas e, por isso, nao aparecem no resultado. Nos modos da Solucao 2, as 5 chaves do arquivo foram carregadas na RAM antes da ordenacao.

8. Trechos representativos da implementacao

```
// Busca binaria iterativa
while (left < right) {
    mid = left + (right - left) / 2;
    if (v[mid] == target) return 1;
    if (v[mid] < target) left = mid + 1;
    else right = mid;
}

// Insercao apenas quando a chave nao foi encontrada
exists = find_key_sorted(&vec, mode, key, &local_cmp);
if (exists) {
    (*duplicates)++;
    continue;
}

pos = lower_bound(vec.data, vec.size, key);
...
csv_append_row(dest, line);
vector_insert_sorted(&vec, key, pos);
```

9. Conclusao

As quatro implementacoes foram desenvolvidas e testadas em C. Na Solucao 1, foram implementadas as buscas sequencial iterativa e recursiva sobre o arquivo nao ordenado. Na Solucao 2, o MergeSort iterativo e recursivo organiza as chaves na RAM, permitindo usar as buscas binaria iterativa e recursiva.

Os testes mostraram que somente os identificadores que ainda nao existiam foram adicionados ao final do arquivo. A analise tambem mostra a diferenca entre as buscas sequencial e binaria, alem do uso maior de memoria e do custo de manter o vetor ordenado na Solucao 2.

10. Estrutura do projeto

```
trabalho_csv_unico/
├── src/
│   ├── csv_utils.h / csv_utils.c
│   ├── sequential_search.h / sequential_search.c
│   ├── mergesort.h / mergesort.c
│   ├── binary_search.h / binary_search.c
│   └── main.c
├── csv_unico.c                # versao standalone
├── data/
│   ├── destino_exemplo.csv
│   └── novas_exemplo.csv
├── docs/complexidade.md
├── Makefile
└── README.md
```

11. Compilacao e execucao

```
make
./csv_unico data/novas_exemplo.csv data/destino_exemplo.csv merge-it
```

12. Fontes consultadas

[1] Portal de Dados Abertos. Conjunto "Compras e contratos".

https://dados.gov.br/dados/conjuntos-dados/compras_contratos

[2] Compras Governamentais - consulta publica de contratos.

https://compras.dados.gov.br/contratos/v1/contratos?cnpj_contratada=05513573000132

[3] Catalogo ODA - Compras Governamentais, documentacao de endpoints e formatos de resposta, incluindo CSV. <https://catalogodedadosabertos.com.br/Comprasgovernamentais>

Repositorio Git publico: <https://github.com/MatheusSZR/atividade-csv-unico>

Observacao: os arquivos CSV em data/ sao usados apenas para os testes do programa. Eles nao substituem o arquivo da fonte oficial, que esta indicado no README e nesta secao.